



網路協定期末專題

Vegas vs Reno



國立彰化師範大學
National Changhua University of Education

2026 年 6 月 21 日

M1454025

陳昶安

內容

選定的 TCP 版本名稱及其壅塞控制(congestion control)機制描述	2
實驗場景及網路拓撲圖描述.....	3
效能圖與說明：	7
心得與其他實作	9
GitHub 連結.....	10

選定的 TCP 版本名稱及其壅塞控制 (congestion control)機制描述

選定 TCP Vegas 。

Vegas 壅塞控制機制描述(參考於 NS3 文件：

https://www.nsnam.org/doxygen/d6/de2/classns3_1_1_tcp_vegas.html

)

TCP Vegas 是一種純粹基於延遲的擁塞控制演算法，它採用主動方案，透過在瓶頸佇列中保持較小的封包積壓(Backlog)來防止丟包，意即主動降低 cwnd 值來減少壅塞發生，BaseRTT 連線過程中觀測到的最小 RTT 值。

Vegas 會持續測量連接的實際吞吐量 (如公式 (1) 所示)，並與公式 (2) 計算的預期吞吐量進行比較。公式 (3) 中這兩個發送速率之間的差異反映了瓶頸處排隊的額外資料包數量。

$$actual = \frac{cwnd}{RTT} \quad (1)$$

$$expected = \frac{cwnd}{BaseRTT} \quad (2)$$

$$diff = expected - actual \quad (3)$$

有三個參數值 alpha, beta, gamma，預設為 2, 4, 1，Vegas 用預期吞吐量

與實際吞吐量的差異 $diff$ 公式(3)來調整 $cwnd$ 值，目標 $\alpha \leq diff \leq \beta$ ，而 γ 與 $diff$ 用於緩啟動切換到壅塞避免上面。

Cwnd 值調整：

1. 如果 $diff < \alpha$ ，網路還沒有明顯壅塞， $cwnd$ 增加。
2. 如果 $\beta < diff$ ，可能壅塞了， $cwnd$ 減少。
3. 如果 $\alpha \leq diff \leq \beta$ 則 $cwnd$ 不變。

實驗場景及網路拓撲圖描述

實驗參數

1. Router 之間(Backbone Network)
 - i. 頻寬： 10Mbps
 - ii. 延遲時間： 20ms
2. Leaf 到 Router
 - i. 頻寬： 15Mbps
 - ii. 延遲時間： 1ms
3. 緩衝區大小(TCP-Linux-Reno 範例原始)：

- i. Sender: 1MB
 - ii. Receiver: 1MB
- 4. 模擬時間： 60 秒
- 5. Queue 大小
 - i. Reno: 38 (packets)
 - ii. Vegas: 100(packets)
- 6. Segment 大小：1460 (MTU = 1500, Header= 40)
- 7. SACK：啟用
- 8. 初始 CWND = 10
- 9. Vegas (3*參考文件預設值):
 - i. Alpha = 6
 - ii. Beta = 12
 - iii. Gamma = 3
- 10. UDP
 - i. 頻寬： 5 Mbps(佔據 Router 一半)
 - ii. 大小： 1 KB(1024)
 - iii. 開始發送：第 15 秒開始

參數說明

為了使 Router 成為 bottleneck 將頻寬設定較小於 Leaf 值以便造成壅塞
進而觀察機制的不同，Buffer Size 依照範例給的值無更動也沒有發生錯誤，模擬時間為了快速實驗設定 60 秒。

原本 Reno queue 也是設定 100，在改變頻寬後發現非常不穩定，所以上網找公式來算：

- $RTT = 2 * (1 + 20 + 1) = 44ms$
- Bottleneck = 10Mbps = 10,000,000 bps
- BDP (Bits) : $10,000,000 \text{ bps} * 0.044 \text{ s} = 440,000 \text{ Bits} = 55,000$

Bytes

- 封包數 (Packets) : $55,000 \text{ Bytes} / 1460 \text{ Bytes (MSS)} \approx 38$ 個封包。

Segment 大小按照乙太網路的 MTU = 1500 減去 TCP Header 40 = 1460 bytes。

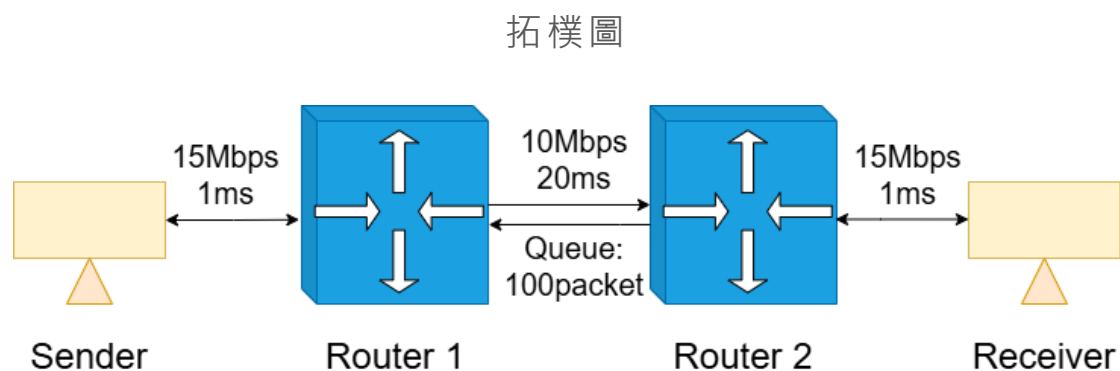
SACK 按照範例啟用使 Reno 可以利用。

CWND 按照範例設定為 10。

Vegas 預設設定 alpha, beta, gamma(2, 4, 1)發現吞吐量離 10Mbps 的理

論值有段距離，所以用 2~5 倍來測試，4 倍幾乎沒有繼續成長，5 倍反而造成延遲(queue size 變動)，所以設定為(8, 16, 4)。

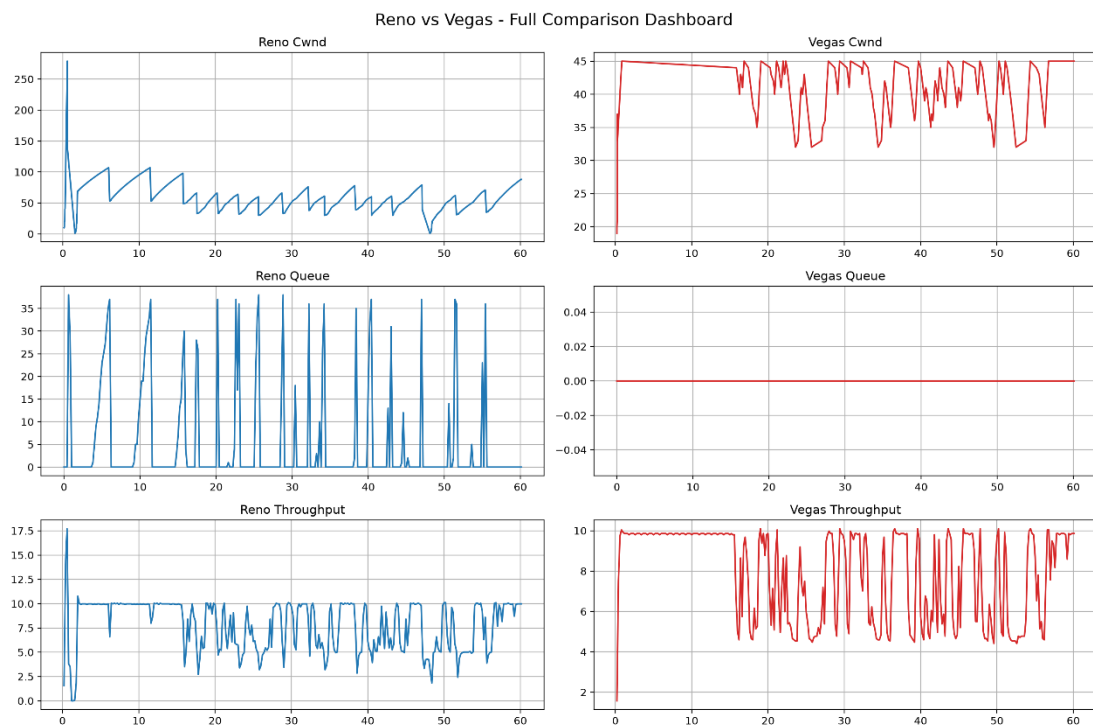
為了改變 throughput 跟 queue，分別加入 UDP 封包干擾，Leaf 頻寬 > Router 頻寬，並從第 15 秒開始產生用來觀測遇到壅塞的狀態跟平常的狀態差異。



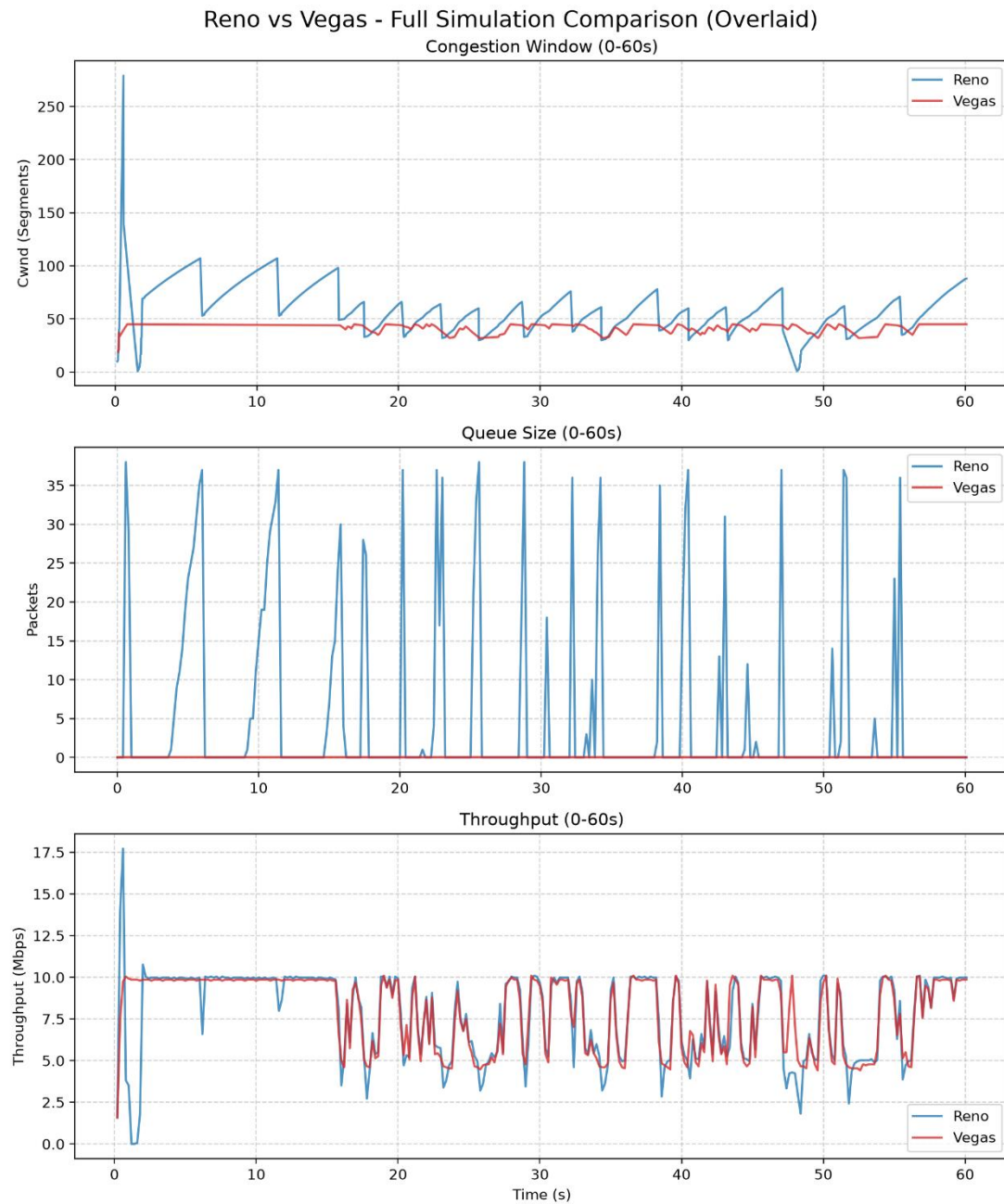
圖一、網路拓樸圖

由圖一、網路拓樸圖可以知道，我們有兩個 Router 作為骨幹網路以及一個傳送端跟一個接收端，參考範例的設計，而效能瓶頸點在於 Router 1 到 Router 2 之間，由於傳送端以 15Mbps, 1ms delay 得頻寬送資料而 Backbone 網路中只有 10Mbps, 20ms delay 的狀況所以用最大頻寬的 cwnd 值的方式傳送的話勢必會造成 queue 的改變與壅塞。

效能圖與說明：



圖二、Reno, Vegas 分開觀察



圖三、Reno, Vegas 結合觀察

畫圖的 Python 程式碼由 Gemini Pro 慷慨贊助，由於 Vegas queue 沒有變動所以事件捕抓為空，所以用補 0 的方式繪製，cwnd 值在前 15 秒穩定時也是用補充的方式來補足缺失。

依據圖二、圖三可以知道，Vegas 在 Alpha, Beta, Gamma 設定為 8, 16, 4 十於環境中吞吐量幾乎與具有 SACK 的 Linux Reno 一樣好，且其探索平穩到理論最高值就停止成長穩定於 10Mbps 左右，而 Reno 則使緩啟動衝過頭遇到 Time-out 而歸並更改 slow start threshold 成一半才達到平衡且有時會有波動。

UDP Packet 在第 15 秒開始出現隨機傳送(On/Off, OnOffHelper)，其頻寬為 5Mbps 相當於 Bottleneck 的一半，可以看到兩者的反應，cwnd 跟 Throughput 開始震盪，但可以看到 Vegas 的幅度較小。

Queue 值 Reno 震盪大，且到第 15 秒以後頻率更加快，而 Vegas 則是始終為 0 的穩定策略，主動的偵測與計算最大值並降低 cwnd 值在此有明顯的優勢。

心得與其他實作

在本次實驗中，最初使用 Ubuntu 做雙系統進行 NS3 3.47 的模擬，後來發現到文書處理有些不方便，而想起強化學習的 TCP 研究做法，最後把 NS3 裝到 Docker 上面使用掛載資料家的方式進行編譯使用，這樣坐起來可移植且簡單使用(CLI 介面)。

NS3 的學習難度感覺頗高，對文件的觀看還未熟悉，初學 157 頁，進階

335 頁，模組庫 739 頁，而單獨從 example 修改程式可以短期完成配合網路搜尋與 AI 問答可以達成，Vegas 論文也還沒看完，稍微有了解事件驅動的意思，其中一開始沒加入 UDP 時 Vegas 甚至只有前面不到 1 秒的時間有 cwnd 值，只能用補充的方式，因為其機制的關係滿意 cwnd 值而環境又沒有變動就抓不到。

最終在裝 Docker 時使用 NS3 3.48 也成功從 tcp-linux-reno 範例改出 linux reno, vegas 的實驗環境，vegas 沒有範例但設計在 src/internet 中有實作所以可以呼叫。

GitHub 連結

https://github.com/WilliamChen2002/NP_Final_Report_NS3_Docker.git